



Module 1 - Text Generation with Amazon Bedrock

This notebook provides sample code with step by step instructions for using Amazon Bedrock's **Converse API**.

Contents

1. [Setup](#)
 2. [Text Summarization with Foundational Models](#)
 - A. [Text Summarization using the Invoke Model API](#)
 - B. [Text Summarization using the Converse API](#)
 - C. [Overview of the Converse API](#)
 - D. [Easily switch between different models](#)
 - E. [Cross-Regional Inference in Amazon Bedrock](#)
 - F. [Multi-turn conversations](#)
 - G. [Streaming Responses with the ConverseStream API](#)
 3. [Code Generation with Foundational Models](#)
 4. [Function Calling with the Converse API](#)
 - A. [4.1 Function Calling Flow](#)
 - B. [Example: Implementation of Weather Function Calling with Bedrock's Converse API](#)
 5. [Conclusion](#)
-

Introduction

Welcome to this introduction to building conversational AI with Amazon Bedrock's Converse API! The primary goal of this chapter is to provide a comprehensive introduction to Amazon Bedrock APIs for generating text. While we'll explore various use cases like summarization and code generation, our focus is on understanding the API patterns.

In this notebook, you will:

1. Learn the basics of the Amazon Bedrock **Invoke API**
2. Explore the more powerful **Converse API** and its features like multi-turn conversation, streaming, or function calling
3. Apply these APIs across various foundation models

4. Compare results across different state-of-the-art models

Pre-requisites

This notebook requires permissions to:

- Access **Amazon Bedrock**.

1. Setup

1.1 Import the required libraries

```
In [ ]: import json
import boto3
import botocore
from IPython.display import display, Markdown
import time
```

1.2 Initial setup for clients, global variables and helper functions

```
In [ ]: # Initialize Bedrock client
session = boto3.session.Session()
region = session.region_name
bedrock = boto3.client(service_name='bedrock-runtime', region_name=region)
```

```
In [ ]: # Define model IDs that will be used in this module
MODELS = {
    "Claude 3.7 Sonnet": "us.anthropic.claude-3-7-sonnet-20250219-v1:0",
    "Claude 3.5 Sonnet": "us.anthropic.claude-3-5-sonnet-20240620-v1:0",
    "Claude 3.5 Haiku": "us.anthropic.claude-3-5-haiku-20241022-v1:0",
    "Amazon Nova Pro": "us.amazon.nova-pro-v1:0",
    "Amazon Nova Micro": "us.amazon.nova-micro-v1:0",
    "Meta Llama 3.1 70B Instruct": "us.meta.llama3-1-70b-instruct-v1:0"
}
```

```
In [ ]: # Utility function to display model responses in a more readable format
def display_response(response, model_name=None):
    if model_name:
        display(Markdown(f"### Response from {model_name}"))
    display(Markdown(response))
    print("\n" + "-"*80 + "\n")
```

2. Text Summarization with Foundation Models

Let's start by exploring how to leverage Amazon Bedrock APIs for text summarization. We'll first use the basic Invoke API, then introduce the more powerful Converse API.

As an example, let's take a paragraph about Amazon Bedrock from an [AWS blog post](#).

```
In [ ]: text_to_summarize = """
AWS took all of that feedback from customers, and today we are excited to announce a new service that makes FMs from AI21 Labs, Anthropic, Stability AI, and Amazon Bedrock is the easiest way for customers to build and scale generative AI-based democratizing access for all builders. Bedrock will offer the ability to access for text and images—including Amazon's Titan FMs, which consist of two new LLMs today—through a scalable, reliable, and secure AWS managed service. With Bedrock, customers can easily find the right model for what they're trying to get done, customize FMs with their own data, and easily integrate and deploy them into their tools and capabilities they are familiar with, without having to manage any infrastructure with Amazon SageMaker ML features like Experiments to test different models and more.
"""
```

2.1 Text Summarization using the Invoke Model API

Amazon Bedrock's **Invoke Model API** serves as the most basic method for sending requests to foundation models. Since each model family has its own distinct request and response format, you'll need to craft specific JSON payloads tailored to each model.

For this example, we will call Claude 3.7 Sonnet via Invoke Model API (using the `invoke_model` function of the Bedrock Runtime Client) to generate a summary of our text.

```
In [ ]: # Create prompt for summarization
prompt = f"""Please provide a summary of the following text. Do not add any information.
<text>
{text_to_summarize}
</text>
"""
```

```
In [ ]: # Create request body for Claude 3.7 Sonnet
claude_body = json.dumps({
    "anthropic_version": "bedrock-2023-05-31",
    "max_tokens": 1000,
    "temperature": 0.5,
    "top_p": 0.9,
    "messages": [
```

```

        {
            "role": "user",
            "content": [{"type": "text", "text": prompt}]
        }
    ],
})

```

```

In [ ]: # Send request to Claude 3.7 Sonnet
try:
    response = bedrock.invoke_model(
        modelId=MODELS["Claude 3.7 Sonnet"],
        body=claude_body,
        accept="application/json",
        contentType="application/json"
    )
    response_body = json.loads(response.get('body').read())

    # Extract and display the response text
    claude_summary = response_body["content"][0]["text"]
    display_response(claude_summary, "Claude 3.7 Sonnet (Invoke Model API)")

except botocore.exceptions.ClientError as error:
    if error.response['Error']['Code'] == 'AccessDeniedException':
        print(f"\x1b[41m{error.response['Error']['Message']}\n"
              "\nTo troubleshoot this issue please refer to the following resource\n"
              "\nhttps://docs.aws.amazon.com/IAM/latest/UserGuide/troubleshoot_ac\n"
              "\nhttps://docs.aws.amazon.com/bedrock/latest/userguide/security-ia\n")
    else:
        raise error

```

2.2 Text Summarization using the Converse API (Recommended Approach)

While the **Invoke Model API** allows direct access to foundation models, it has several limitations:

1. it uses different request/response formats for each model family;
2. there is no built-in support for multi-turn conversations;
3. it requires custom handling for different model capabilities.

The **Converse API** addresses these limitations by providing a unified interface. Let's explore it on our text summarization task:

```

In [ ]: # Create a converse request with our summarization task
converse_request = {
    "messages": [
        {
            "role": "user",
            "content": [
                {

```

```

        "text": f"Please provide a concise summary of the following\n\n{context}\n\n",
    ],
    ],
    },
    "inferenceConfig": {
        "temperature": 0.4,
        "topP": 0.9,
        "maxTokens": 500
    }
}

```

```

In [ ]: # Call Claude 3.7 Sonnet with Converse API
try:
    response = bedrock.converse(
        modelId=MODELS["Claude 3.7 Sonnet"],
        messages=converse_request["messages"],
        inferenceConfig=converse_request["inferenceConfig"]
    )

    # Extract the model's response
    claude_converse_response = response["output"]["message"]["content"][0]["text"]
    display_response(claude_converse_response, "Claude 3.7 Sonnet (Converse API)")
except botocore.exceptions.ClientError as error:
    if error.response['Error']['Code'] == 'AccessDeniedException':
        print(f"\x1b[41m{error.response['Error']['Code']}: {error.response['Error']['Message']}\n")
        print("Please ensure you have the necessary permissions for Amazon Bedrock")
    else:
        raise error

```

2.3 Overview of the Converse API

Now, that we have used the **Converse API**, let's take some time to take a closer look. To use the Converse API, you use the [Converse](#) or [ConverseStream](#) (for streaming responses) operations to send messages to a model. While, it is possible to use the existing base inference operations ([InvokeModel](#) or [InvokeModelWithResponseStream](#)) for conversation applications as well, we recommend using the Converse API as it provides consistent API, that works with [all Amazon Bedrock models that support messages](#). This means you can write code once and use it with different models. Should a model have unique inference parameters, the Converse API also allows you to pass those unique parameters in a model specific structure. You can use the Amazon Bedrock Converse API to create conversational applications that send and receive messages to and from an Amazon Bedrock model. For example, you can create a chat bot that maintains a conversation over many turns and uses a persona or tone customization that is unique to your needs, such as a helpful technical support assistant. The Converse API also supports other Bedrock capabilities, like [tool use](#) and [guardrails](#).

Let's break down its key components (you can also review the [documentation](#) for a full list of parameters):

```
{
  "modelId": "us.anthropic.claude-3-7-sonnet-20250219-v1:0", //
  Required: Model identifier

  "messages": [ // Required: Conversation history
    {
      "role": "user", // Who sent the message
      "content": [
        {
          "text": "Your prompt or message here" // Message content
        }
      ]
    }
  ],

  "system": [ // Optional: System instructions
    {
      "text": "You are a helpful AI assistant."
    }
  ],

  "inferenceConfig": { // Optional: Inference parameters
    "temperature": 0.7, // Randomness (0.0-1.0)
    "topP": 0.9, // Diversity control (0.0-1.0)
    "maxTokens": 2000, // Maximum response length
    "stopSequences": [] // Stop generation triggers
  },

  "toolConfig": { // Optional: Function calling setup
    "tools": [],
    "toolChoice": {
      "auto": {} // Let model decide when to use tools
    }
  }
}
```

2.4 Easily switch between models

One of the biggest advantages of the Converse API is the ability to easily switch between models using the exact same request format. Let's compare summaries across different foundation models by looping over the model dictionary we defined above:

```
In [ ]: # call different models with the same converse request
results = {}
```

```

for model_name, model_id in MODELS.items(): # looping over all models defined
    try:
        start_time = time.time()
        response = bedrock.converse(
            modelId=model_id,
            messages=converse_request["messages"],
            inferenceConfig=converse_request["inferenceConfig"] if "inferenceConfig" in converse_request else {}
        )
        end_time = time.time()

        # Extract the model's response using the correct structure
        model_response = response["output"]["message"]["content"][0]["text"]
        response_time = round(end_time - start_time, 2)

        results[model_name] = {
            "response": model_response,
            "time": response_time
        }

        print(f"✅ Successfully called {model_name} (took {response_time} seconds)")

    except Exception as e:
        print(f"❌ Error calling {model_name}: {str(e)}")
        results[model_name] = {
            "response": f"Error: {str(e)}",
            "time": None
        }

```

```

In [ ]: # Display results in a formatted way
for model_name, result in results.items():
    if "Error" not in result["response"]:
        display(Markdown(f"### {model_name} (took {result['time']} seconds)"))
        display(Markdown(result["response"]))
    print("-" * 80)

```

2.5 Cross-Regional Inference in Amazon Bedrock

Amazon Bedrock offers Cross-Regional Inference which automatically selects the optimal AWS Region within your geography to process your inference requests. Cross-Regional Inference offers higher throughput limits (up to 2x allocated quotas) and seamlessly manages traffic bursts by dynamically routing requests across multiple AWS regions, enhancing application resilience during peak demand periods without additional routing or data transfer costs. Customers can control where their inference data flows by selecting from a pre-defined set of regions, helping them comply with applicable data residency requirements and sovereignty laws. Moreover, this capability prioritizes the connected Bedrock API source region when possible, helping to minimize latency and improve responsiveness. As a result, customers can enhance their applications' reliability, performance, and

efficiency. Please review the list of [supported regions and models for inference profiles](#).

To use Cross-Regional Inference, you simply need to specify a cross-region inference profile as the `modelId` when making a request. Cross-region inference profiles are identified by including a region prefix (e.g., `us.` or `eu.`) before the model name.

For example:

```
{
  "Amazon Nova Pro": "amazon.nova-pro-v1:0", # Regular model ID
  "Amazon Nova Pro (CRIS)": "us.amazon.nova-pro-v1:0" #
Cross-regional model ID
}
```

Let's see how easy it is to use Cross Region Inference by invoking the Claude 3.5 Sonnet model:

```
In [ ]: # Regular model invocation (standard region)
standard_response = bedrock.converse(
    modelId="anthropic.claude-3-5-sonnet-20240620-v1:0", # Standard model ID
    messages=converse_request["messages"]
)

# Cross-region inference (note the "us." prefix)
cris_response = bedrock.converse(
    modelId="us.anthropic.claude-3-5-sonnet-20240620-v1:0", # Cross-region model ID
    messages=converse_request["messages"]
)

# Print responses
print("Standard response:", standard_response["output"]["message"]["content"])
print("Cross-region response:", cris_response["output"]["message"]["content"])
```

2.6 Multi-turn Conversations

The Converse API makes multi-turn conversations simple. Let's see it in action:

```
In [ ]: # Example of a multi-turn conversation with Converse API
multi_turn_messages = [
    {
        "role": "user",
        "content": [{"text": f"Please summarize this text: {text_to_summarize}"}],
    },
    {
        "role": "assistant",
        "content": [{"text": results["Claude 3.7 Sonnet"]["response"]}],
    },
]
```



```

    {
        "role": "user",
        "content": [{"text": "Can you make this summary even shorter, just 1 s
    }
]

try:
    response = bedrock.converse(
        modelId=MODELS["Claude 3.7 Sonnet"],
        messages=multi_turn_messages,
        inferenceConfig={"temperature": 0.2, "maxTokens": 500}
    )

    # Extract the model's response using the correct structure
    follow_up_response = response["output"]["message"]["content"][0]["text"]
    display_response(follow_up_response, "Claude 3.7 Sonnet (Multi-turn conver

except Exception as e:
    print(f"Error: {str(e)}")

```

2.7 Streaming Responses with ConverseStream API

For longer generations, you might want to receive the content as it's being generated. The ConverseStream API supports streaming, which allows you to process the response incrementally:

```

In [ ]: # Example of streaming with Converse API
def stream_converse(model_id, messages, inference_config=None):
    if inference_config is None:
        inference_config = {}

    print("Streaming response (chunks will appear as they are received):\n")
    print("-" * 80)

    full_response = ""

    try:
        response = bedrock.converse_stream(
            modelId=model_id,
            messages=messages,
            inferenceConfig=inference_config
        )
        response_stream = response.get('stream')
        if response_stream:
            for event in response_stream:

                if 'messageStart' in event:
                    print(f"\nRole: {event['messageStart']['role']}")

                if 'contentBlockDelta' in event:
                    print(event['contentBlockDelta']['delta']['text'], end="")

```

```

        if 'messageStop' in event:
            print(f"\nStop reason: {event['messageStop']['stopReason']}")

        if 'metadata' in event:
            metadata = event['metadata']
            if 'usage' in metadata:
                print("\nToken usage")
                print(f"Input tokens: {metadata['usage']['inputTokens']}")
                print(f"Output tokens: {metadata['usage']['outputTokens']}")
                print(f"Total tokens: {metadata['usage']['totalTokens']}")
            if 'metrics' in event['metadata']:
                print(f"Latency: {metadata['metrics']['latencyMs']} milliseconds")

    print("\n" + "-" * 80)
    return full_response

except Exception as e:
    print(f"Error in streaming: {str(e)}")
    return None

```

```

In [ ]: # Let's try streaming a longer summary
streaming_request = [
    {
        "role": "user",
        "content": [
            {
                "text": f"""Please provide a detailed summary of the following

{text_to_summarize}

Make your summary comprehensive but clear.
"""
            }
        ]
    }
]

```

```

In [ ]: # Only run this when you're ready to see streaming output
streamed_response = stream_converse(
    MODELS["Claude 3.7 Sonnet"],
    streaming_request,
    inference_config={"temperature": 0.4, "maxTokens": 1000}
)

```

3. Code Generation with FMs

Foundation models like Claude excel not only at generating prose but also at

creating programming language code, making them valuable tools for engineering work automation. In this chapter, we'll use Claude and the Bedrock Converse API to generate code for a weather function that we'll in the next chapter integrate as a tool for our models. We'll craft a specific prompt for Claude, configure and execute a Bedrock Converse API request, and save the generated weather function with mock data for use in the next chapter.

Prompt the LLM to write us a `get_weather` function

```
In [ ]: code_generation_prompt = """
Create a Python function called get_weather that accepts a location as parameter.
The function should return a dictionary containing weather data (condition, temperature, etc.).
Use a mock data structure instead of actual API calls. Include New York, San Francisco, and London.
The return statement should look like the following: return weather_data.get(location, {}).
Only return the function and no preamble or examples.
"""
```

Create a converse request with our code generation task

```
In [ ]: converse_request = {
    "messages": [
        {
            "role": "user",
            "content": [
                {
                    "text": f"{code_generation_prompt}"
                }
            ]
        }
    ],
    "inferenceConfig": {
        "temperature": 0.0,
        "topP": 0.9,
        "maxTokens": 500
    }
}
```

Call Claude 3.7 Sonnet with Converse API

```
In [ ]: try:
    response = bedrock.converse(
        modelId=MODELS["Claude 3.7 Sonnet"],
        messages=converse_request["messages"],
        inferenceConfig=converse_request["inferenceConfig"]
    )

    # Extract the model's response
    claude_converse_response = response["output"]["message"]["content"][0]["text"]
    display_response(claude_converse_response, "Claude 3.7 Sonnet (Converse API)")
except botocore.exceptions.ClientError as error:
```

```
if error.response['Error']['Code'] == 'AccessDeniedException':
    print(f"\x1b[41m{error.response['Error']['Code']}: {error.response['Er
    print("Please ensure you have the necessary permissions for Amazon Bed
else:
    raise error
```

Now let's copy the created function to the below cell and define it in our notebook.

```
In [ ]: # Copy-Paste the created get_weather in this cell:
# def get_weather(location): ....
```

4. Function Calling with Amazon Bedrock

Modern LLMs like Claude go beyond generating free-form text — they can also reason about when external tools or functions should be used to better answer user questions. This capability, known as *function calling* (or *tool use*), enables the model to decide *which* function to call, *when* to call it, and *what* parameters to provide — but importantly, the model does not execute the function itself.

Instead, the model returns a well-structured response (typically in JSON format) that describes the intended function call. It's then up to your application to detect this output, execute the requested function (such as calling an API or querying a database), and pass the result back to the model — allowing it to generate a final, user-friendly response that incorporates real-world data.

Function calling is especially useful when building LLM-powered applications that need access to dynamic, external information — for example, retrieving real-time weather data, which is exactly what we'll demonstrate in this section.

4.1 Function Calling Flow

Amazon Bedrock natively supports function calling through its Converse API, which provides a consistent way to give LLMs access to tools or functions that live outside the model.

The typical function calling flow looks like this:

- **Step 1 — Function Recognition.** When the model (e.g., Claude 3.7) identifies that a tool should be used to answer a user query (e.g., "What's the weather in Seattle?"), it returns a structured response indicating:

- The function name to call (e.g., `get_weather`)

- The required input parameters (e.g., `location="Seattle"`)
- **Step 2 — Function Execution** (by your application). The application is responsible for:
 - Executing the requested function
 - Capturing the output (e.g., current weather data)
 - Passing the result back to the model
- **Step 3 — Final Response Generation.** The LLM uses the function's output to generate a natural language response for the user.

4.2 Example: Implementation of Weather Function Calling with Bedrock's Converse API

In this example, we'll integrate a simple `get_weather` function with Claude 3.7 using the Converse API. The tool specification we provide to Claude defines:

- The function name: `get_weather`
- The function's purpose: *"Retrieve weather for a given location"*
- The required input parameters: `location` (string)

Example Workflow

Important: Make sure you created the `get_weather()` function in one of the cells above (by copy-pasting the code that Claude has generated). In case something went wrong you can uncomment the back up implementation of the weather lookup function below:

```
In [ ]: # def get_weather(location):
#       """Mock function that would normally call a weather API"""
#       print(f"Looking up weather for {location}...")

#       # In a real application, this would call a weather API
#       weather_data = {
#           "New York": {"condition": "Partly Cloudy", "temperature": 72, "humidity": 65},
#           "San Francisco": {"condition": "Foggy", "temperature": 58, "humidity": 85},
#           "Miami": {"condition": "Sunny", "temperature": 85, "humidity": 75},
#           "Seattle": {"condition": "Rainy", "temperature": 52, "humidity": 90}
#       }
```

```
#     return weather_data.get(location, {"condition": "Unknown", "temperature":
```

Now, lets define our tool specification..

```
In [ ]: weather_tool = {
    "tools": [
        {
            "toolSpec": {
                "name": "get_weather",
                "description": "Get current weather for a specific location",
                "inputSchema": {
                    "json": {
                        "type": "object",
                        "properties": {
                            "location": {
                                "type": "string",
                                "description": "The city name to get weather f
                            }
                        },
                        "required": ["location"]
                    }
                }
            },
            "toolChoice": {
                "auto": {} # Let the model decide when to use the tool
            }
        }
    ],
    "inferenceConfig": {
        "temperature": 0.0, # Use 0 temperature for deterministic function ca
        "maxTokens": 500
    }
}
```

.. and create a simple function calling request

```
In [ ]: function_request = {
    "messages": [
        {
            "role": "user",
            "content": [
                {
                    "text": "What's the weather like in San Francisco right no
                }
            ]
        }
    ],
    "inferenceConfig": {
        "temperature": 0.0, # Use 0 temperature for deterministic function ca
        "maxTokens": 500
    }
}
```

With the **tool specification** and the **function_request** we can now invoke the model and take a look at its response. Observe how it correctly fills out the

"toolUse" parameter in its response.

```
In [ ]: response = bedrock.converse(
    modelId=MODELS["Claude 3.7 Sonnet"],
    messages=function_request["messages"],
    inferenceConfig=function_request["inferenceConfig"],
    toolConfig=weather_tool
)
print(json.dumps(response, indent=2))
```

Let's now build a function to handle the complete function calling flow

```
In [ ]: def handle_function_calling(model_id, request, tool_config):
    try:
        # Step 1: Send initial request
        response = bedrock.converse(
            modelId=model_id,
            messages=request["messages"],
            inferenceConfig=request["inferenceConfig"],
            toolConfig=tool_config
        )

        # Check if the model wants to use a tool (check the correct response s
        content_blocks = response["output"]["message"]["content"]
        has_tool_use = any("toolUse" in block for block in content_blocks)

        if has_tool_use:
            # Find the toolUse block
            tool_use_block = next(block for block in content_blocks if "toolUse"
            tool_use = tool_use_block["toolUse"]
            tool_name = tool_use["name"]
            tool_input = tool_use["input"]
            tool_use_id = tool_use["toolUseId"]

            # Step 2: Execute the tool
            if tool_name == "get_weather":
                tool_result = get_weather(tool_input["location"])
            else:
                tool_result = {"error": f"Unknown tool: {tool_name}"}

            # Step 3: Send the tool result back to the model
            updated_messages = request["messages"] + [
                {
                    "role": "assistant",
                    "content": [
                        {
                            "toolUse": {
                                "toolUseId": tool_use_id,
                                "name": tool_name,
                                "input": tool_input
                            }
                        }
                    ]
                }
            ]
```

```

    },
    {
        "role": "user",
        "content": [
            {
                "toolResult": {
                    "toolUseId": tool_use_id,
                    "content": [
                        {
                            "json": tool_result
                        }
                    ],
                    "status": "success"
                }
            }
        ]
    }
]

# Step 4: Get final response
final_response = bedrock.converse(
    modelId=model_id,
    messages=updated_messages,
    inferenceConfig=request["inferenceConfig"],
    toolConfig=tool_config
)

# Extract text from the correct response structure
final_text = ""
for block in final_response["output"]["message"]["content"]:
    if "text" in block:
        final_text = block["text"]
        break

return {
    "tool_call": {"name": tool_name, "input": tool_input},
    "tool_result": tool_result,
    "final_response": final_text
}
else:
    # Model didn't use a tool, just return the text response
    text_response = ""
    for block in content_blocks:
        if "text" in block:
            text_response = block["text"]
            break

    return {
        "final_response": text_response
    }

except Exception as e:
    print(f"Error in function calling: {str(e)}")

```



```
return {"error": str(e)}
```

Execute the function calling flow with Claude 3.7 Sonnet

```
In [ ]: function_result = handle_function_calling(
        MODELS["Claude 3.7 Sonnet"],
        function_request,
        weather_tool
    )

# Display the results
if "error" not in function_result:
    if "tool_call" in function_result:
        print(f"Tool Call: {function_result['tool_call']['name']}({function_re
        print(f"Tool Result: {function_result['tool_result']}")

    display_response(function_result["final_response"], "Claude 3.7 Sonnet (Fu
else:
    print(f"Error: {function_result['error']}")
```

5. Conclusion

In this notebook, we've explored various capabilities of Amazon Bedrock APIs for the following use cases:

1. **Text Summarization:** Using both Invoke and Converse APIs to create concise summaries
2. **Model Flexibility:** Demonstrating easy switching between different foundation models
3. **Multi-turn Conversations:** Building contextual dialogues using the Converse API
4. **Streaming Responses:** Implementing real-time content delivery with ConverseStream API
5. **Code Generation:** Using LLMs to generate functional code
6. **Function Calling:** Integrating external tools to retrieve weather information for specific locations

You are now prepared to move on to chapter 2, Knowledge Bases and RAG. Have fun!